REACT JS
CRASH COURSE

Expanded Topics, Practical Examples & Project Folder Guide

Core learning areas	Practical work
React fundamentals, ES6, Hooks, routing, TypeScript, forms and validation	React UI connected to Express APIs and PostgreSQL/Neon, Redux Toolkit, authentication and e-commerce

Prepared from the training notes and project paths supplied.

This expanded guide organizes the recorded topics into a learner-friendly course outline. Folder paths are retained where they help identify the practical examples. The guide is a structured coverage record, not an independent verification that every topic was completed by every learner.

Trainer reference edition • October 2026

1. Course Overview & Learning Path

The notes describe a practical React-focused full-stack course that begins with web and JavaScript foundations, moves through React components and Hooks, and then connects the frontend to APIs, databases, state management, forms, routing and authentication.

Suggested learning sequence

- **Stage 1 — Foundations:** HTML basics, development tools, React setup, ES6 syntax, imports/exports and styling.
- **Stage 2 — React programming:** functional components, JSX, state, events and the major built-in Hooks.
- **Stage 3 — Reusable UI:** custom Hooks, controlled/uncontrolled forms, portals and reusable modal patterns.
- **Stage 4 — Data integration:** Express endpoints, API calls, PostgreSQL/Neon and user CRUD.
- **Stage 5 — Application architecture:** Redux Toolkit, multiple slices, TypeScript, routing and protected pages.
- **Stage 6 — Forms and security:** React Hook Form, Zod, JWT-based authentication and cookie-focused flows.
- **Stage 7 — Capstone:** e-commerce application with a payment-gateway component.

What learners should be able to demonstrate

- Build a React interface from reusable functional components.
- Explain why state updates cause a component to render again and how Effects are used.
- Choose suitable Hooks for state, references, context, reducers and performance-related tasks.
- Call backend endpoints and present database-backed data in a React UI.
- Organize shared state with Redux Toolkit and inspect changes using Redux DevTools.
- Create validated forms, navigate between pages and protect authenticated areas.
- Explain the high-level frontend/backend/authentication flow in a full-stack application.

Important note

The supplied notes record folder names, examples and topics. They do not prove that every item was completed, assessed or mastered. Use the final checklist near the end to confirm actual classroom coverage.

2. Development Setup, HTML & JavaScript Foundations

The early folders and notes cover introductory screens, editor/browser extensions, HTML basics, serving a simple site, React installation and ES6 fundamentals.

Topics recorded in the notes

- HTML basics and a simple website served locally.
- Introductory development screens and useful VS Code / Chrome extensions.
- React application setup and initial project exploration.
- Functional components compared with class components.
- ES6 fundamentals, arrow functions, import/export and Flexbox.
- React styling, including a Sass-related installation note.

Practical folders

```
E:\MURALI\REACT-JS-CRASH-COURSE\html-basics
E:\MURALI\REACT-JS-CRASH-COURSE\my-site
E:\MURALI\REACT-JS-CRASH-COURSE\testapp
E:\MURALI\REACT-JS-CRASH-COURSE\react-course\src-Installation
E:\MURALI\REACT-JS-CRASH-COURSE\usa-reactjs\my-react-app
```

Local static-site example

The notes show installing and running `http-server` from a site folder. The purpose is to serve static files locally and open them in a browser. This is separate from running a Vite development server for a React project.

Suggested teaching emphasis

- Explain the roles of HTML, CSS and JavaScript before introducing React.
- Practice declaring variables, arrow functions, template literals, destructuring and module imports/exports.
- Explain how a React component returns JSX and how JavaScript expressions are embedded in JSX.
- Distinguish a component's source code from the browser-rendered UI.

Folders highlighted in the notes

```
E:\MURALI\REACT-JS-CRASH-COURSE\react-course\src-Installation
E:\MURALI\REACT-JS-CRASH-COURSE\src-Lesson-1-basics
E:\MURALI\REACT-JS-CRASH-COURSE\src - M1-Lesson-1-ES6
E:\MURALI\REACT-JS-CRASH-COURSE\src - M1-Lesson-1-ES6-Fundamentals
E:\MURALI\REACT-JS-CRASH-COURSE\src - M1-Lesson-1-ImportExport
E:\MURALI\REACT-JS-CRASH-COURSE\src - M1-Lesson-1-FlexBox
E:\MURALI\REACT-JS-CRASH-COURSE\src - M3-Lesson-2-Styling In React
```

3. React Components, JSX, State & Rendering

The notes emphasize functional components and the relationship between JavaScript logic, component state and rendered HTML content.

Core concepts

- **Functional components:** JavaScript functions that return JSX and describe a portion of the user interface.
- **JSX:** syntax that lets component code describe UI while embedding JavaScript expressions.
- **State:** data owned by a component that can change over time and influence what is displayed.
- **State updates:** React schedules and may batch updates; code should not assume the state variable changes immediately after a setter call.
- **Component reuse:** separate UI and behavior into components that can be composed together.
- **Class components:** included as a comparison point, particularly around lifecycle methods.

Practical examples recorded

```
E:\my-react-app\src-Lesson-1-basics
```

```
E:\my-react-app\src - M3-Lesson-2-Styling In React
```

Suggested classroom exercises

- Build a small component that displays a title, description and a list.
- Move repeated UI into a reusable component and pass information through props.
- Create a counter with state and demonstrate that changing a normal local variable does not, by itself, trigger a React render.
- Use browser React Developer Tools to inspect component hierarchy and props/state where available.

Checkpoint questions

- What does a component return?
- What is the difference between a normal variable and state?
- Why is JSX not the same thing as an HTML file?
- When would you extract a repeated UI block into its own component?

Trainer note

Use small examples first. Make learners predict the UI after each state update, then run the code and compare the result.

4. useState and useEffect

The supplied notes identify useState and useEffect as key teaching topics and point to a dedicated practical folder.

useState — component state

useState lets a functional component retain data between renders. The setter requests a state update, after which React renders the component with the updated value. Typical teaching examples include counters, toggles, input fields and simple collections.

useEffect — synchronizing with external systems

useEffect is used for work that needs to happen in relation to a render, such as setting up a subscription, interacting with an external API or synchronizing with browser APIs. The dependency array controls when the Effect is re-run; cleanup can release resources when the Effect is replaced or the component unmounts.

Key distinctions to teach

- A state setter requests an update; it does not mutate the existing variable in place.
- Effects are not required for every calculation. Values derived from props/state can often be calculated during render.
- Dependencies should match the reactive values used by an Effect.
- Repeated API calls can arise from dependency mistakes or development behavior; diagnose the cause rather than suppressing Effects blindly.
- When fetching data, consider loading, success and error states.

Practical folders and examples

```
E:\my-react-app\src -M3-Lesson-4-useState and useEffect-IMP
E:\my-react-app\src-Lesson-3-React-Hooks-NICE
E:\my-react-app\src - M3-Lesson-4-All Hooks-1
```

Files mentioned in the notes

```
App-useState-1.jsx, App-useState-2.jsx, App-useState-3.jsx
App-useEffect.jsx
```

Practice sequence

- Build a counter and a show/hide toggle with useState.
- Control a form input using state.
- Add an Effect that logs or synchronizes with an external value.
- Add cleanup to an interval or event listener example.
- Review dependency changes and identify why an Effect runs.

5. useRef, useMemo & useCallback

The notes contain several examples for references, memoization and callback identity, including comparisons with and without memoization.

useRef

useRef stores a mutable value that persists across renders without causing a render when it changes. A common example is referencing a DOM element to focus an input or measure an element. It can also hold values that are not directly part of the UI.

Examples named in the notes

```
App-useRef-1.jsx through App-useRef-5.jsx
```

```
ImperativeHandleExample.jsx, CustomInput.jsx
```

Forwarding refs and imperative handles

The notes specifically mention useImperativeHandle and forwardRef. Teach these as a way to expose a limited imperative interface from a child component when a declarative prop-based approach is not sufficient.

Portals and modal UI

```
App-useRefCreatePortals.jsx, App-useRefCreatePortalsPersonalInfo.jsx
```

```
App-Reusable-Modal-Component-In-body.jsx
```

```
App-Reusable-Modal-Component-In-modal-container.jsx
```

The notes show a separate `modal-root` element in index.html and examples using createPortal. A portal renders UI into a different DOM location while retaining its place in the React component tree.

useMemo

useMemo caches the result of a calculation between renders when its dependencies have not changed. It is useful for expensive calculations or when stable object/array identity matters to a measured optimization. It should not be added automatically to every calculation.

```
App-useMemo-WITHOUT.jsx, App-useMemo-WITH.jsx, App-useMemo-Vs-useEffect.jsx
```

useCallback

useCallback caches a function definition between renders when dependencies remain unchanged. It can help when passing callbacks to memoized children or when a function identity is a meaningful dependency.

```
App-useCallback-WITHOUT.jsx, App-useCallback-WITH.jsx, App-useCallback-1.jsx
```

6. useContext, useReducer & useLayoutEffect

The training notes list context, reducer and layout-effect examples, including a combined context/reducer pattern.

useContext — sharing values through a component tree

Context can help avoid passing the same data through many intermediate components. The notes explicitly recommend teaching the prop-drilling example first, then moving to useContext.

```
src - M10-Lesson-3-Prop-Drilling-useContext() Hook
App-useContext.jsx
```

Teaching sequence

- Start with a parent passing props through multiple levels.
- Identify the intermediate components that do not use the value themselves.
- Introduce a context provider and read the value in a descendant.
- Discuss that context is useful for shared values, but is not automatically the best solution for every state problem.

useReducer — state transitions through a reducer

useReducer organizes state updates around a reducer function and dispatched actions. It is useful when several related state transitions need a clear structure, such as a multi-step form or a collection with add/update/delete actions.

```
App-useReducer-1.jsx through App-useReducer-5-EXAMPLE
App-useReducer-6-COMBINED-useContext-useReducer
```

useLayoutEffect

useLayoutEffect runs after React commits DOM changes but before the browser repaints. It is mainly useful for layout measurement or synchronously adjusting layout to avoid visible flicker. For ordinary synchronization, useEffect is generally the better default.

```
App-useLayoutEffect-1.jsx, App-useLayoutEffect-2.jsx
```

Related component patterns

```
App-6-Controlled-Component.jsx
App-6-Uncontrolled-Component.jsx
App-7-HOC-Higherordercomponents-1.jsx through App-7-HOC-Higherordercomponents-4.jsx
```

The notes also list Higher-Order Components (HOCs). Use these examples to discuss a component-wrapping pattern for sharing behavior, and compare it with custom Hooks where appropriate.

7. Custom Hooks & Reusable Logic

A dedicated custom-Hooks folder contains examples for local storage, counters, toggles, fetching and HTTP CRUD operations.

Why custom Hooks?

A custom Hook extracts reusable stateful logic from a component. Its name begins with `use`, and it can call other Hooks while following the Rules of Hooks. It shares logic, not a single shared state instance by default: each component calling a custom Hook normally gets its own Hook state.

Custom Hooks recorded in the notes

- **useLocalStorage:** synchronize a value with browser local storage.
- **useCounter:** package counter state and actions into a reusable interface.
- **useToggle:** manage a boolean toggle with a small API.
- **useFetch:** organize data-fetching behavior.
- **useHttp:** encapsulate HTTP requests and CRUD-related behavior.

Main folder

```
E:\my-react-app\src - M10-Lesson-4-Custom Hooks
```

Example files mentioned

```
App-1.useLocalStorageCustomHook.jsx, Counter.jsx, Counter1.jsx, Counter2.jsx
App-2.useCounterCustomHook.jsx, useCounter.js
App-3.useToggleCustomHook.jsx, ToggleComponent.jsx, useToggle.js
App-4.useFetchCustomHook.jsx, PostsList.jsx, useFetch.js
App-5.useHttp-CRUDCustomHooks.jsx, UsersList.jsx, UpdateUser.jsx, DeleteUser.jsx, useHttp.js
```

Suggested practical exercise

- Implement a small useToggle Hook and consume it in two components.
- Create a useCounter Hook with increment, decrement and reset operations.
- Create a data-fetching Hook that exposes data, loading and error states.
- Use an HTTP Hook from a list, update and delete component, then discuss where request-specific logic belongs.

Design checkpoint

A custom Hook should make a component easier to read. If it merely renames one line or hides important behavior without improving reuse or clarity, it may not be worth extracting.

8. Node.js, Express & API Integration

The notes include simple Express endpoints, a users endpoint backed by JSONPlaceholder, and React frontend examples that connect to backend APIs.

Backend folders and files

```
E:\test-node-api
E:\backend-rtk
server-1-simple-endpoint.js — Hello Express
server-2-users-endpoints.js — JSONPlaceholder users
E:\my-react-app\src-backendapi
```

What this module connects

- An Express server exposes endpoints over HTTP.
- The React frontend sends requests to those endpoints.
- The backend returns JSON data and status codes.
- The frontend updates its UI to represent loading, success and failure states.
- CORS and environment configuration may be relevant when frontend and backend run on different origins.

Suggested endpoint practice

- Create a simple health or greeting endpoint.
- Create a users endpoint returning a JSON array.
- Call the endpoint from React and render the returned users.
- Handle request failures without leaving the UI in a permanent loading state.
- Compare frontend routes with backend API routes; they serve different purposes.

Useful debugging workflow

- Use the browser Network tab to inspect request URL, method, status, response and timing.
- Check the server terminal for route errors and runtime exceptions.
- Verify that JSON response shape matches the data the UI expects.
- Check the browser console for rejected promises and rendering errors.

Checkpoint

A React component should not assume every HTTP request succeeds. Teach students to model loading, empty, success and error states explicitly.

9. PostgreSQL / Neon & Database-Backed React

The notes record a Node/Express backend connected to Neon PostgreSQL, plus React examples both without Redux Toolkit and with Redux Toolkit.

Project folders

```
E:\users-api
E:\my-react-app\src-WITHOUT-REDUX-Toolkit-neondb
E:\my-react-app\src-REDUX-Toolkit-neondb
```

Recorded frontend examples

```
App-COUNTER-USERS.jsx
App-COUNTER.jsx
App-USERS.jsx
App-WITH-SWEETALERT-ICONS.jsx
```

Typical request/data flow

- The user interacts with the React page.
- React calls an Express API endpoint.
- Express runs a parameterized SQL query through the PostgreSQL driver.
- The server returns a JSON response.
- React updates the list, form or status message based on the response.

Database security and configuration

The original notes contained a live-looking database connection string. It has intentionally not been reproduced in this document. Keep DATABASE_URL in a server-side environment file, exclude secrets from source control and never expose database credentials in browser code or public teaching notes. If a real credential was shared outside a trusted environment, rotate it.

Suggested CRUD learning objectives

- Read records and display them in a table or list.
- Create a record using a form.
- Update an existing record and reflect the result in the UI.
- Delete a record with clear user feedback.
- Distinguish database errors, validation errors and network errors.

Feedback tools

The notes list SweetAlert2 for alert/dialog feedback and lucide-react for icons. These improve user feedback and visual affordances; they do not replace server-side validation or error handling.

10. Redux Toolkit, Multiple Slices & DevTools

The notes list Redux Toolkit projects, a multiple-slices frontend/backend example and a TypeScript CRUD project.

Folders recorded

```
E:\my-react-app\src-REDUX-Toolkit
E:\my-react-app\src-ReduxToolkitMultiSlices
E:\backendMultiSlicesPostgresql
E:\redux-typescript-crud
```

Core Redux Toolkit pieces

- **configureStore:** creates the Redux store and combines reducers.
- **createSlice:** groups a slice's initial state, reducer logic and generated action creators.
- **Provider:** makes the store available to the React component tree.
- **useSelector:** reads data from the Redux store.
- **useDispatch:** dispatches actions to request state changes.

What to demonstrate

- A counter slice with increment/decrement/reset actions.
- A users slice that represents users and CRUD-related state.
- Multiple slices combined in one store.
- A component reading state with useSelector and dispatching actions with useDispatch.
- How an API-backed flow differs from a local-only counter example.

Redux DevTools vs React Developer Tools

The notes distinguish the two browser extensions: React Developer Tools is useful for inspecting the React component tree and profiling, while Redux DevTools is useful for inspecting store state, dispatched actions, action history and state changes.

Suggested explanation for students

The Redux store holds application state; reducers describe how state changes in response to actions. Components do not directly edit the store object. They dispatch actions, and the reducer logic produces the next state.

Checkpoint questions

- What is the role of Provider?
- When should state be local to a component rather than global?
- What does createSlice generate?
- How can Redux DevTools help trace a bug?

11. TypeScript with React

The supplied notes include a React TypeScript demo, event typing, preventDefault() investigation, and a Redux Toolkit CRUD project in TypeScript.

Project folders

```
E:\react-typescript-demo
E:\redux-typescript-crud
```

Topics in the notes

- React event types and event handlers.
- The relationship between the HTML element, the handler and the event object.
- TypeScript React and React DOM type packages.
- Compiler options and explicit TypeScript checks.
- TypeScript in a Redux Toolkit CRUD application.

Understanding event types

The HTML element identifies where the event originates; the React handler identifies the interaction being handled; the event type describes the event object the handler receives. For example, form submission, input changes and button clicks use different event types.

preventDefault()

The notes include a browser-console exercise to inspect a submit event and confirm that preventDefault is a method. In a React form handler, preventDefault() is commonly used to stop the browser's default form submission navigation while the application handles the form in JavaScript.

TypeScript checklist

- Give component props clear types.
- Use suitable event types for form, input and button handlers.
- Keep type-only imports explicit when compiler settings require them.
- Understand the difference between a compile-time type error and a runtime error.
- Use strictness and no-unused checks to catch issues earlier.

Practice tasks

- Type a form component with a submit handler.
- Type an input change handler and a select handler.
- Create typed props for a reusable button or form field.
- Trace a TypeScript error to the relevant component or event declaration.

12. React Hook Form & Zod Validation

The notes identify JSX and TSX versions of React Hook Form with Zod and list assignment files, including examples with and without useForm context.

Project folders and files

```
E:\react-rhk-zod
E:\typescriptapp-rhk-zod
AssignmentZODForm.tsx
DemoFormWithZOD.tsx
AssignmentZODForm-without-useForm-Context.tsx
AssignmentZODForm-with-useForm-Context.tsx
```

Concepts to explain

- **React Hook Form:** manages form values, submission and field-level form state with a focus on avoiding unnecessary re-renders.
- **Zod schema:** describes the expected shape and validation rules for submitted data.
- **Resolver:** connects Zod validation to React Hook Form.
- **TypeScript input/output types:** can distinguish values as entered from values after schema transformations.
- **Context-based form patterns:** can make form methods available to nested fields without passing every method through props.

Typical validation scenarios

- Required name and email fields.
- Email-format validation.
- Numeric fields with coercion and range constraints.
- Clear field-specific validation messages.
- Submission only after validation passes.

Suggested classroom sequence

- Start with a simple form and display submitted values.
- Add a Zod schema and required-field rules.
- Add numeric validation and explain how input strings become numbers.
- Compare the context and non-context form examples.
- Test empty fields, malformed email, invalid numbers and valid submissions.

Important distinction

Client-side validation improves user experience but is not a security boundary. The backend must independently validate incoming data before writing to the database.

13. React Router: Pages, Parameters & Protected Routes

The notes contain a dedicated routing assignment and lesson folders covering setup, nested routes, route parameters, redirects and private/protected routes.

Folders highlighted

```
E:\pagecomponent-app
E:\my-react-app\src-ROUTING-Assignment
src - M11-Lesson-1-Setting up React Router
src - M11-Lesson-1-Deeply Nested Routes with Siblings
src - M11-Lesson-2-Route Parameters
src - M11-Lesson-3-Nested Routes
src - M11-Lesson-4-Redirects
src - M11-Lesson-4-PrivateProtectedRoutes
```

Terms and APIs recorded

- BrowserRouter, Routes, Route, path and element.
- Link and NavLink for navigation.
- useParams for reading route parameters.
- useNavigate for programmatic navigation and passing navigation state.
- useLocation for reading the current location and navigation state.
- Outlet for rendering nested child routes.

Teaching sequence

- Create a few page components and map them to routes.
- Navigate with Link and NavLink; discuss active links.
- Add a dynamic route parameter and read it with useParams.
- Build nested routes and render child pages with Outlet.
- Demonstrate redirects and a protected route wrapper.
- Explain that frontend route protection controls navigation/UI but must be paired with backend authorization.

Page vs component

A page is typically a component selected for a route, while a component can be any reusable piece of UI. A route can render a page component, and that page can compose many smaller components.

Checkpoint

Ask learners to draw the route tree and identify which route renders each URL. This is often easier to understand than reading nested JSX without a diagram.

14. JWT Authentication, Cookies & Authorization

The notes mark two projects as teaching targets for JWT authentication and authorization, with an explicit emphasis on HTTP-only cookies.

Projects marked for teaching

```
E:\Advaitecs\pern-auth-system
E:\Advaitecs\jwt-cookie-teaching-demo
```

Concepts to cover

- **Authentication:** establishing who the user is, usually through a login process.
- **Authorization:** checking what an authenticated user is allowed to do.
- **JWT:** a signed token format that can carry claims; a token is not automatically encrypted just because it is signed.
- **HTTP-only cookie:** a browser cookie that client-side JavaScript cannot read directly.
- **Cookie security options:** Secure, SameSite, expiry/max-age and path should be configured for the deployment context.
- **Backend enforcement:** protected APIs must verify identity and permissions on the server.

Suggested flow diagram to teach

Login form → **Express API** → verify credentials → issue session/token in an appropriate cookie → browser sends cookie on later requests → backend validates request → endpoint returns authorized data.

Security discussion points

- Do not place database credentials or signing secrets in frontend code.
- HTTP-only reduces JavaScript access to the cookie but does not by itself solve every cross-site request risk.
- Configure CORS and cookie settings together when frontend and backend origins differ.
- Use HTTPS in production and avoid logging tokens or credentials.
- Distinguish token expiry, logout, invalid token and insufficient permissions.

Scope note

The notes label E:\Advaitecs\jwt-cookies as not required and list two other folders as needing a decision. Those folders are not presented here as confirmed course requirements.

15. E-commerce Capstone, UI Feedback & Debugging

The notes identify a React TypeScript e-commerce project with a payment gateway as a final project area, and mention SweetAlert2, Lucide icons and browser developer tools.

Capstone folder

```
E:\MURALI\REACT-JS-CRASH-COURSE\ReactTSEcomApp
```

Potential integration checklist

- Reusable product, cart, checkout and confirmation UI components.
- React state or a state-management approach for cart information.
- Form validation before checkout.
- Backend endpoints for products and order-related operations, where implemented.
- Clear loading, success and error feedback.
- Payment-gateway integration and verification of payment status on the server side.

Libraries mentioned in the notes

- **SweetAlert2:** dialog/alert feedback for success, error or confirmation.
- **lucide-react:** icon components for buttons and UI status.
- **React Developer Tools:** component inspection and profiling.
- **Redux DevTools:** store state and action history inspection.

Debugging workflow

- Console: inspect errors, warnings and logged values.
- Network: verify requests, response payloads, status codes and failed calls.
- React Developer Tools: inspect component structure and props/state.
- Redux DevTools: inspect actions and state transitions.
- Backend terminal: inspect server-side exceptions and database failures.

Payment safety reminder

Do not treat a frontend success message or redirect as proof that payment succeeded. Verify payment status with the provider/server flow before marking an order as paid.

16. Folder Index & Project Map

This index groups the main folders from the supplied notes so the trainer can quickly locate practical examples during revision. Paths are transcribed from the notes where possible.

Area	Folder / project	Use
Foundations	E:\MURALI\REACT-JS-CRASH-COURSE\html-basics	HTML basics
Foundations	E:\MURALI\REACT-JS-CRASH-COURSE\my-site	Static site / http-server
React basics	E:\my-react-app	Main React practice workspace
Hooks	src -M3-Lesson-4-useState and useEffect-IMP	State and Effects
Hooks	src-Lesson-3-React-Hooks-NICE	Built-in Hooks examples
Custom Hooks	src - M10-Lesson-4-Custom Hooks	Reusable Hook examples
Express	E:\backend-rtk	Express endpoints
API frontend	E:\my-react-app\src-backendapi	React API calls
PostgreSQL	E:\users-api	Express + Neon DB
Redux	E:\my-react-app\src-REDUX-Toolkit	Redux Toolkit
Redux multi-slice	E:\my-react-app\src-ReduxToolkitMultiSlices	Multiple slices
TypeScript	E:\react-typescript-demo	React + TypeScript
Forms	E:\react-rhk-zod	React Hook Form + Zod JSX
Forms TSX	E:\typescriptapp-rhk-zod	React Hook Form + Zod TSX
Routing	E:\pagecomponent-app	Pages and components
Authentication	E:\Advaitecs\pern-auth-system	JWT auth teaching project
Authentication	E:\Advaitecs\jwt-cookie-teaching-demo	Cookie-focused JWT demo
Capstone	E:\MURALI\REACT-JS-CRASH-COURSE\ReactTSEcomApp	E-commerce/payment gateway

Folder names with spaces or unusual punctuation have been preserved where practical. Confirm the exact local path in File Explorer if a folder does not open.

17. Final Brush-up & Coverage Checklist

Use this page during the final revision session. Tick an item only after the learner can explain it or demonstrate it independently.

Done	Area	Can explain / demonstrate
■	**Foundations**	HTML basics, development tools, React setup, ES6, imports/exports and styling.
■	**Components**	Functional components, JSX, props, state and re-rendering.
■	**Hooks**	useState, useEffect, useRef, useMemo, useCallback, useContext, useReducer and useLayoutEffect.
■	**Reusable patterns**	Custom Hooks, controlled/uncontrolled components, portals/modals and HOCs.
■	**Backend APIs**	Express endpoints, JSON responses, HTTP errors and React API calls.
■	**Database**	PostgreSQL/Neon-backed CRUD and server-side environment configuration.
■	**Redux Toolkit**	Store, slices, Provider, useSelector, useDispatch and DevTools.
■	**TypeScript**	Props, event types, form events, compiler checks and typed CRUD.
■	**Forms**	React Hook Form, Zod schema rules, resolver, validation errors and TSX types.
■	**Routing**	Links, parameters, nested routes, redirects, Outlet and protected routes.
■	**Authentication**	Authentication vs authorization, JWT, HTTP-only cookies and server-side checks.
■	**Capstone**	E-commerce flow, form validation, API integration, payment status and user feedback.
■	**Debugging**	Console, Network, React Developer Tools, Redux DevTools and backend logs.

Final trainer reminder

The notes contain a broad set of practical examples across React and full-stack development. This expanded guide organizes those examples into a coherent review document. Confirm actual coverage with the class checklist, and update the document if any additional modules or assignments were completed.

Security hygiene: do not share database URLs, passwords, JWT secrets or API keys in course notes, screenshots or public repositories.